

Deklaracja projektowa		
Temat projektu:	Politechnika Białostocka: Gra o Przetrwanie, satyryczna, wieloosobowa gra planszowa online (mechanika handlu nieruchomościami) osadzona na kampusie PB i w kultowych miejscach Białegostoku. Walutą są Złotówki (PLN).	
Github	https://github.com/kszontek/Politechnika_Monopoly	
Skład grupy		
Imię i nazwisko	Nr indexu	PS
Piotr Czyżewski	117344	9
Bartosz Składanowski	117206	9
Mikołaj Staniaszek	117260	9
Maciej Socik	117245	9
Zakres funkcjonalności		
MVP(Must Have Features)	Nice to Have Features	
- Rejestracja i logowanie (Spring Security) z podziałem na role: Admin, Moderator, Użytkownik oraz Gość (podgląd rozgrywki bez logowania). - Wieloosobowa rozgrywka online w czasie rzeczywistym (WebSockets) - synchronizacja ruchów pionków i rzutów kostką. - Dynamiczna plansza w HTML5 / Tailwind CSS Grid z płynnymi animacjami przesunięć pionków (bez Canvas). - Transakcyjny silnik rozliczeń (kupno obiektów, opłata za pole rywala) oparty na procedurach Oracle PL/SQL. - Zapis stanu gry w HttpSession; statystyki i historia zrzucane do bazy przy wylogowaniu. - Widok oparty na dacie bieżącej: „Codzienne Koło Fortuny Stypendialnego” (Daily Login Bonus & Streak). - Panel CRUD dla Admina/Moderatora: doładowanie środków (PLN) na konta graczy oraz weryfikacja użytkownika po przestaniu legitymacji PB.	- Efekty dźwiękowe w formacie MP3 (np. brzęk monet przy zysku). - System codziennych osiągnięć (Daily Achievements, np. „Stały Bywalec Gwintu”). - Chat graczy w pokoju rozgrywki działający przez WebSockets. - Losowe awatary studenckie pobierane z zewnętrznego API za pomocą Klienta REST.	

Do czego REST będzie używane i dlaczego jego użycie ma sens w kontekście projektu.	
Backend udostępnia publiczny endpoint GET /api/ranking-najbogatszych zwracający JSON z globalnym rankingiem majątku graczy (PLN) i liczbą wygranych partii – do statystyk w menu głównym. REST służy też do asynchronicznej walidacji stanów kont (np. przy zakupie nieruchomości) oraz weryfikacji formularzy rejestracyjnych. Użycie REST ma sens, bo oddziela warstwę danych i statystyk od logiki gry czasu rzeczywistego (WebSockets) i pozwala udostępniać dane niezależnie od trwającej sesji rozgrywki.	
Jakie encje będą obsługiwane przez operacje CRUD oraz jak pasuje to do projektu.	
CRUD obejmuje encję Użytkownicy/Studenti. Admin i Moderator zarządzają kontami graczy: doładowują środki (PLN) oraz weryfikują konto po przestaniu skanu legitymacji PB (zmiana statusu na zweryfikowany). Admin ma pełne uprawnienia (m.in. usuwanie i edycja wszystkich kont), Moderator działa w węższym zakresie. Pasuje to do projektu, bo pozwala bezpiecznie zarządzać finansami w grze i dopuszczać do rozgrywki wyłącznie potwierdzonych studentów PB.	
Jakie będą mechanizmy obsługi formularzy oraz walidacji danych wejściowych użytkownika.	
Formularze rejestracji i logowania obsługiwane są w warstwie Spring Security. Walidacja danych wejściowych odbywa się po stronie serwera z użyciem Spring Validation (adnotacje @Min, @Size i podobne) oraz dodatkowo po stronie klienta. Dane wejściowe (login, hasło, kwoty transakcji) są weryfikowane pod kątem poprawności i kompletności przed zapisem do bazy Oracle, co zabezpiecza spójność rozgrywki.	
Przewidywane trudności algorytmiczne:	
– Synchronizacja stanu gry w czasie rzeczywistym między wieloma graczami (WebSockets) i unikanie konfliktów jednoczesnych ruchów. – Bezpieczne, atomowe transakcje PLN między graczami w odizolowanym bloku transakcyjnym PL/SQL oraz obsługa algorytmu bankructwa przy braku środków. – Poprawne wyliczanie „Daily Streak” przez trigger porównujący daty kolejnych logowań (inkrementacja licznika vs reset). – Logika kart wydarzeń wpływających globalnie na planszę (mnożniki opłat, przenoszenie pionków, efekty czasowe trwające kilka tur).	